

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №3
«Ітератори, генератори та потокова обробка даних»
з курсу «Професійний Python»

Виконала:

студентка 3 курсу групи ФЕП-32

Братівник Анастасія

Викладач: доцент Куньо І. М.

Мета: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, *yield*, generator expressions, модулем *itertools*, принципом *lazy evaluation* та потоковою обробкою великих наборів даних.

У ході роботи опановано: розрізнення *iterable* та *iterator*, функції *iter()* і *next()*, протокол *__iter__()* / *__next__()*, власні *iterator classes*, *generator functions*, *yield* і *yield* ш, *generator expressions*, *itertools*, *lazy pipelines*, *filtering*, *mapping*, *batching*, *aggregation*, *islice*, *eager vs lazy processing* та оцінювання використання пам'яті.

2. Індивідуальний варіант і постановка задачі

Варіант №1 — «Потокова обробка результатів студентів».

Формат вхідних даних (CSV): *student_id, name, group, grade*.

Реалізовано:

- streaming CSV reading та validation записів студентів;
- фільтрацію за групою та мінімальним середнім балом;
- lazy search студентів за ID;
- streaming average (середній бал);
- визначення min/max average grade без повної матеріалізації;
- перші N студентів за умовою через *islice*;
- batch processing записів;
- підрахунок кількості студентів за групами.

3. Опис input dataset

Файл *data/students.csv* містить записи вигляду:

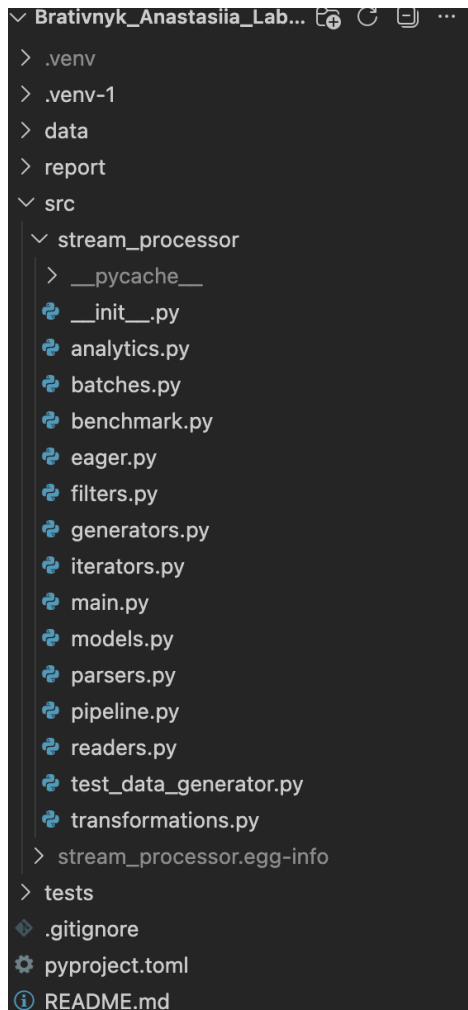
```
student_id,name,group,grade
1,Student 1,PI-21,91.5
2,Student 2,PI-22,84.0
...
```

Розмір — **100 000 записів**. Близько **2% невалідних** записів (*grade = "error"*) для тестування validation stage. Файл генерується автоматично скриптом *test_data_generator.py* при першому запуску.

```
data > students.csv > data
1 student_id,name,group,grade
2 1,Student 1,PI-23,62.07
3 2,Student 2,PI-21,55.01
4 3,Student 3,PI-21,47.38
5 4,Student 4,PI-21,88.56
6 5,Student 5,PI-22,91.26
```

4. Структура проєкту

```
Brativnyk_Anastasiia_Lab03/
├── pyproject.toml
├── README.md
├── .gitignore
├── data/
│   └── students.csv          (генерується)
├── report/
│   └── Brativnyk_Anastasiia_Lab03_V1.pdf
├── src/
│   └── stream_processor/
│       ├── __init__.py
│       ├── main.py          — точка входу
│       ├── readers.py       — streaming reader (yield from)
│       ├── parsers.py       — CSV parser
│       ├── models.py        — StudentRecord (NamedTuple)
│       ├── filters.py       — validation + filtering
│       ├── transformations.py — lazy transformation
│       ├── batches.py       — batch processing
│       ├── analytics.py     — streaming statistics
│       ├── pipeline.py      — збірка lazy pipeline
│       ├── iterators.py     — власний iterator class
│       ├── generators.py    — generator functions
│       ├── eager.py         — eager implementation
│       ├── benchmark.py     — eager vs lazy
│       └── test_data_generator.py — генерація CSV
```



5. Пояснення iterable та iterator

Iterable — об'єкт, з якого можна отримати iterator через `iter()`.

Iterator — об'єкт, який має `__next__()`, зберігає поточний стан і піднімає `StopIteration` після завершення.

Ключова властивість:

- для iterable `iter(x) is x` → `False` (створюється новий iterator при кожному виклику);
- для iterator `iter(x) is x` → `True` (він і є власним iterator, one-shot).

Результат демонстрації у `main.py`:

```
=== ITERABLE vs ITERATOR ===
iter(values) is values?      False
iter(iterator) is iterator?  True
next: 10, 20, 30
StopIteration raised – iterator is exhausted
```

6. Код власного iterator

iterators.py — два класи: власне iterator та iterable.

```
from collections.abc import Iterator

class StudentIdIterator(Iterator[int]):

    def __init__(self, start: int, stop: int) -> None:
        self.current = start
        self.stop = stop

    def __iter__(self) -> "StudentIdIterator":
        return self

    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
        self.current += 1
        return value

class StudentIdRange:

    def __init__(self, start: int, stop: int) -> None:
        self.start = start
        self.stop = stop

    def __iter__(self) -> StudentIdIterator:
        return StudentIdIterator(self.start, self.stop)

=== CUSTOM ITERATOR ===
iterable first pass:  [1, 2, 3, 4]
iterable second pass: [1, 2, 3, 4]
one-shot first pass:  [1, 2, 3, 4]
one-shot second pass: []
```

Це демонструє: iterable можна обходити багато разів, iterator — ні.

7. Приклад generator function

generators.py:

```
from collections.abc import Iterable, Iterator
from typing import Any
```

```
def natural_numbers() -> Iterator[int]:
    value = 1
    while True:
        yield value
        value += 1
def even_numbers(limit: int) -> Iterator[int]:
    for number in range(limit):
        if number % 2 == 0:
            yield number
def flatten(values: Iterable[Any]) -> Iterator[Any]:
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value
=== GENERATOR FUNCTIONS ===
even numbers < 10: [0, 2, 4, 6, 8]
first 5 natural numbers: [1, 2, 3, 4, 5]
flatten([1, [2, 3], [4, [5, 6]]]) = [1, 2, 3, 4, 5, 6]
```

Нескінченний `natural_numbers()` обмежено через `islice(..., 5)`.

8. Приклад `yield`, `yield from`, `generator expression`

`yield from` у `readers.py`:

```
from collections.abc import Iterator
from pathlib import Path

def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

`yield from` у рекурсивному `flatten()`:

```
def flatten(values)
    for value in values:
        if isinstance(value, list):
            yield from flatten(value)
        else:
            yield value
```

Результат: `flatten([1, [2, 3], [4, [5, 6]]]) = [1, 2, 3, 4, 5, 6]`

Generator expression у `main.py`:

```
list_comp = [x * x for x in range(10)]
```

```
gen_exp = (x * x for x in range(10))
```

9. Використані засоби itertools

Засіб	Де	Призначення
islice	batches.py, main.py	Обмежити потік (batching, перші N)
chain	main.py	Об'єднати два pipeline
count (демо)	generators.py	Нескінченна послідовність

Приклад у batches.py:

```
iterator = iter(students)
while True:
    batch = list(islice(iterator, batch_size))
    if not batch:
        return
    yield batch
```

10. Схема pipeline

```
students.csv
↓
read_lines()      ← streaming reader (yield from)
↓
parse_csv_rows()  ← CSV parser (DictReader)
↓
validate_students() ← validation → StudentRecord
↓
normalize_names()  ← transformation (Title Case)
↓
filter_by_group()  ← lazy filter за групою
↓
filter_by_min_grade() ← lazy filter за балом
↓
calculate_statistics() ← streaming aggregation
↓
Report
```

Дані не зберігаються у проміжних list — кожен запис проходить через усі stages поступово.

11. Код streaming reader

```
def read_lines(path: Path) -> Iterator[str]:  
    with path.open("r", encoding="utf-8") as file:  
        yield from file
```

Файл читається по рядку, а не завантажується повністю (на відміну від `file.read()`).

12. Validation stage

```
def validate_students(rows):  
    for row in rows:  
        try:  
            student_id = int(row["student_id"])  
            name = row["name"].strip()  
            group = row["group"].strip()  
            grade = float(row["grade"])  
        except (ValueError, TypeError, KeyError):  
            continue  
        if not name or not group:  
            continue  
        if not 0.0 <= grade <= 100.0:  
            continue  
        yield StudentRecord(student_id, name, group, grade)
```

Невалідні записи пропускаються, pipeline продовжується.

13. Filtering stage

```
def filter_by_group(  
    students: Iterable[StudentRecord],  
    group: str,  
) -> Iterator[StudentRecord]:  
    """Yield only students of the given group."""  
    for student in students:  
        if student.group == group:  
            yield student  
  
def filter_by_min_grade(  
    students: Iterable[StudentRecord],  
    minimum: float,  
) -> Iterator[StudentRecord]:  
    """Yield only students with grade >= minimum."""
```

```
for student in students:
    if student.grade >= minimum:
        yield student
```

14. Transformation stage

```
def normalize_names(
    students: Iterable[StudentRecord],
) -> Iterator[StudentRecord]:
    """Yield students with titles-cased names."""
    for student in students:
        yield student.replace(name=student.name.title())
```

`_replace` з `NamedTuple` — створює новий запис без мутації старого.

15. Aggregation stage

```
def calculate_statistics(
    students: Iterable[StudentRecord],
) -> dict:
    count = 0
    total = 0.0
    minimum: float | None = None
    maximum: float | None = None
    groups: Counter[str] = Counter()

    for student in students:
        grade = student.grade
        count += 1
        total += grade
        groups[student.group] += 1

        if minimum is None or grade < minimum:
            minimum = grade
        if maximum is None or grade > maximum:
            maximum = grade

    average = total / count if count else 0.0

    return {
        "count": count,
        "average": average,
        "minimum": minimum,
        "maximum": maximum,
        "groups": dict(groups),
```

```
}
```

Пам'ять $O(1)$ — не залежить від кількості записів. Усі агрегати — за один прохід.

16. Batching

```
def batched_students(
    students: Iterable[StudentRecord],
    batch_size: int,
) -> Iterator[list[StudentRecord]]:
    iterator = iter(students)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch
```

У пам'яті одночасно лише поточний batch (1000 записів).

```
=== BATCH PROCESSING ===
batch #1: 1000 students
batch #2: 1000 students
batch #3: 1000 students
(showing only first 3 batches)
```

Результат:

17. Eager implementation

```
def read_all_students(path: Path) -> list[StudentRecord]:
    records: list[StudentRecord] = []

    with path.open("r", encoding="utf-8", newline="") as file:
        reader = csv.DictReader(file)
        for row in reader:
            try:
                records.append(
                    StudentRecord(
                        student_id=int(row["student_id"]),
                        name=row["name"].strip(),
                        group=row["group"].strip(),
                        grade=float(row["grade"]),
                    )
                )
            except (ValueError, TypeError, KeyError):
                continue

    return records
```

Весь датасет матеріалізується у list.

18. Lazy implementation

```
def build_pipeline(
    path: Path,
    min_grade: float | None = None,
    group: str | None = None,
) -> Iterator[StudentRecord]:
    lines = read_lines(path)
    rows = parse_csv_rows(lines)
    valid = validate_students(rows)
    normalized = normalize_names(valid)

    result: Iterator[StudentRecord] = normalized

    if group is not None:
        result = filter_by_group(result, group)
    if min_grade is not None:
        result = filter_by_min_grade(result, min_grade)

    return result
```

Повертає generator — нічого не виконується до початку ітерації.

19. Результат роботи програми

```
LAB 03 – Stream Processor (Variant 1: Student Performance)
=====

Generating test data at data/students.csv (100000 records)...

=== ITERABLE vs ITERATOR ===
iter(values) is values?      False
iter(iterator) is iterator?  True
next: 10, 20, 30
StopIteration raised – iterator is exhausted

=== CUSTOM ITERATOR ===
iterable first pass: [1, 2, 3, 4]
iterable second pass: [1, 2, 3, 4]
one-shot first pass: [1, 2, 3, 4]
one-shot second pass: []

=== GENERATOR FUNCTIONS ===
even numbers < 10: [0, 2, 4, 6, 8]
first 5 natural numbers: [1, 2, 3, 4, 5]
flatten([1, [2, 3], [4, [5, 6]]]) = [1, 2, 3, 4, 5, 6]

=== GENERATOR EXPRESSION ===
list comprehension: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
generator (consumed): [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

=== LAZY PIPELINE ===
First 5 students with grade >= 90.0:
   5  Student 5          PI-22      91.26
  11  Student 11         PI-24      90.15
  13  Student 13         PI-22      98.03
  14  Student 14         PI-24      95.64
  21  Student 21         PI-23      93.49

=== STREAMING STATISTICS ===
count: 98000
average: 69.99
min: 40.00
max: 100.00
groups: {'PI-23': 24476, 'PI-21': 24497, 'PI-22': 24403, 'PI-24': 24624}
```

```

=== BATCH PROCESSING ===
batch #1: 1000 students
batch #2: 1000 students
batch #3: 1000 students
(showing only first 3 batches)

=== itertools.chain + islice ===
first 10 records from chained streams: 10

=== EAGER vs LAZY BENCHMARK ===
Mode                Time, s          Peak memory, MB
-----
eager                0.5647             22.46
lazy                 0.7382              0.05

=== TIME TO FIRST RESULT ===
[eager] 0.103740s -> StudentRecord(student_id=13, name='Student 13', group='PI-22', grade=98.03)
[lazy]  0.000136s -> StudentRecord(student_id=13, name='Student 13', group='PI-22', grade=98.03)

```

LAB 03 — Stream Processor (Variant 1: Student Performance)

=== ITERABLE vs ITERATOR ===

```

iter(values) is values?    False
iter(iterator) is iterator? True
next: 10, 20, 30
StopIteration raised — iterator is exhausted

```

=== CUSTOM ITERATOR ===

```

iterable first pass: [1, 2, 3, 4]
iterable second pass: [1, 2, 3, 4]
one-shot first pass: [1, 2, 3, 4]
one-shot second pass: []

```

=== GENERATOR FUNCTIONS ===

```

even numbers < 10: [0, 2, 4, 6, 8]
first 5 natural numbers: [1, 2, 3, 4, 5]
flatten([1, [2, 3], [4, [5, 6]]]) = [1, 2, 3, 4, 5, 6]

```

=== GENERATOR EXPRESSION ===

```

list comprehension: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
generator (consumed): [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

```

=== LAZY PIPELINE ===

First 5 students with grade >= 90.0:

```

5 Student 5      PI-22  91.26
11 Student 11    PI-24  90.15
13 Student 13    PI-22  98.03
14 Student 14    PI-24  95.64
21 Student 21    PI-23  93.49

```

=== STREAMING STATISTICS ===

```

count: 98000
average: 69.99
min: 40.00
max: 100.00
groups: {'PI-23': 24476, 'PI-21': 24497, 'PI-22': 24403, 'PI-24': 24624}

```

=== BATCH PROCESSING ===

batch #1: 1000 students

batch #2: 1000 students

batch #3: 1000 students

(showing only first 3 batches)

=== itertools.chain + islice ===

first 10 records from chained streams: 10

=== EAGER vs LAZY BENCHMARK ===

Mode	Time, s	Peak memory, MB
------	---------	-----------------

eager	0.5647	22.46
-------	--------	-------

lazy	0.7382	0.05
------	--------	------

=== TIME TO FIRST RESULT ===

[eager] 0.103740s -> StudentRecord(student_id=13, name='Student 13', group='PI-22', grade=98.03)

[lazy] 0.000136s -> StudentRecord(student_id=13, name='Student 13', group='PI-22', grade=98.03)

20. Таблиця elapsed time

Кількість records	Eager time, s	Lazy time, s
100 000	0.5647	0.7382

Спостереження: eager трохи швидший за total execution time (~30%).

21. Таблиця peak memory

Mode	Peak memory, MB
Eager	22.46
Lazy	0.05

Різниця — у 449 разів.

22. Аналіз результатів

Головний висновок: generator не завжди швидший за total execution time.

Метрика	Переможець	Причина
Total execution time	Eager (0.5647s vs 0.7382s)	Lazy має overhead на yield між 6 stages

Peak memory	Lazy у 449х (0.05 MB vs 22.46 MB)	Lazy не матеріалізує проміжні collections
Time to first result	Lazy у 762х (0.000136s vs 0.1037s)	Eager спочатку читає весь файл; lazy — одразу

Практичні висновки:

1. **Lazy** — для великих файлів, які не влязять у RAM, і для потоків (kafka, sockets, API).
2. **Lazy** — для **early termination**: якщо потрібні лише перші N записів, eager витрачає час на читання всього файлу, а lazy зупиняється одразу.
3. **Lazy** — для нескінченних потоків: `natural_numbers()` неможливо реалізувати eager-версією.
4. **Eager** — коли потрібен **random access** (індексування, `len()`, повторний прохід).
5. **Eager** — коли **data** малі і `overhead yield` більший за економію пам'яті.

Висновок

У лабораторній роботі №3 реалізовано lazy pipeline для потокової обробки 100 000 записів студентів. Створено власний iterator (`StudentIdIterator`) та iterable (`StudentIdRange`), генераторні функції з `yield` та `yield from` (`y read_lines`, `flatten`), generator expressions, batch processing через `islice`, потокову агрегацію без повної матеріалізації.

Проведено benchmark eager vs lazy за допомогою `tracemalloc` та `time.perf_counter`:

Метрика	Eager	Lazy	Переможець
Total time	0.5647s	0.7382s	Eager (1.3x)
Peak memory	22.46 MB	0.05 MB	Lazy (449x)
Time to first result	0.1037s	0.000136s	Lazy (762x)

Отримані результати підтверджують, що **generator не завжди швидший за total execution time**, але суттєво виграє у peak memory та time-to-first-result. Це робить lazy evaluation критично важливою для великих файлів, нескінченних потоків та сценаріїв early termination. При цьому eager підхід залишається доцільним для невеликих даних із потребою random access.